Lecture VI - Modules and the Standard Library

Programming with Python

Dr. Tobias Vlćek



Checkpoint 3

Sessions I-V. The first 40 minutes are the checkpoint. It starts now, before the investor sits down.

- **Individual work**: no AI, no neighbors, no messaging
- Slides, the cheatsheet and your own notebooks **may be open**
- The **link is on Moodle**. Open it and start
- ~6 short tasks: write code, trace code, fix a bug, answer a multiple choice
- It sweeps **everything from Sessions I-V**: variables, control flow, functions, data structures, errors

When you're done

Cmd/Ctrl+S, then menu → *Download* → *Download Python code* → upload the `.py` to the **"Checkpoint 3"** assignment on Moodle, before the 40 minutes are up. **No retakes**: one sitting.

> **Note**
> The live checks under the tasks are **provisional**. The final grading runs on my side. Everything in it was rehearsed in the labs.

EP 6: Due Diligence

The investor walks in

On to Episode 6.

An hour ago the **investor** walked into the shop unannounced. It's **due-diligence week**: before she signs anything, she wants to see how this place actually runs. Tobi offered her a coffee and a spreadsheet he "mostly trusts."

She didn't drink the coffee. She walked to the whiteboard, uncapped a marker, and wrote one question:

> **"Why is everything built from scratch?"**

Part II: the rules change

For five sessions you built everything by hand, on purpose. From today, that changes.

- **AI is now allowed and taught.** We work *with* it, deliberately, starting in today's lab.
- **Disclosure comes with the project:** in Part III you list which AI tools you used and what for. Labs and checkpoints need no note.
- **The course chatbot** (sidebar widget) now gives you **full code** on request, where it used to stop at hints.

For five sessions you worked without AI. From today you may use it, and you stay responsible for every line you hand in.

What is an LLM?

The course chatbot and every AI tool you will use run on one:

- A **Large Language Model** is an **advanced pattern-recognition system**. It has read enormous amounts of text: books, websites, and code
- What it actually does: **predict the next piece of text**, like the word suggestions on your phone keyboard, but far more capable
- **Not like a search engine**: it does not look anything up, it generates from the patterns it learned
- It does not "think" or "understand" the way you do, and it has no feelings to hurt

The model sees tokens, not words

- Text is broken into **tokens**, chunks of text mapped to numbers
- Common words are one token; rarer words split into pieces: "un-", "break", "-able"
- The Mistral models we use have a vocabulary of about **131,000** different tokens
- That is also why famous models fail at "count the r's in *strawberry*": they see a token like `straw` plus a token like `berry`, never the letters

It guesses the next token

Question: "The students opened their ___"

- Books? Laptops? Presents? Eyes?
- Every answer is plausible. None is "the" answer
- The model faces exactly this choice at every token it writes: it picks a **likely** next token, not *the* next token
- So the same question twice can get two different answers. We return to that idea today in *Rehearsing luck*, when the shop's numbers get a seed

The context window (1)

- Everything the model knows about you sits in the **context window**: the text it can hold at once, its short-term memory

- How much is that? A page of prose is about 500 tokens. Mistral's newest model, **Mistral Large 4** (October 2026), holds up to **1 million** tokens at once: about 2,000 pages

The context window (2)

- Larger window: it remembers more of the conversation.
- Smaller: it forgets earlier turns
- Even with a huge window, it **overlooks details**: the longer the chat, the more often a buried detail gets missed
- **Nothing outside the window exists for the model**: not your screen, not your intent

> Tip
> This is why Session I told you: ask the chatbot as specific as possible. Your question fills the window; the model can only use what is in it.

Two accounts this week

Both are free, both are on the AI Tools page, together about ten minutes:

- **Mistral:** a free account for **Vibe**, Mistral's assistant (older guides call it Le Chat). The same account later powers the AI agent inside your editor
- **Zed student plan:** sign in with the GitHub account you made in Session I (it's old enough now) and apply with your KLU e-mail for a free year of Zed Pro

> ! Important
> Turn **off** both training-data switches in Mistral's privacy settings before you paste coursework (the AI Tools page shows where). Nothing here needs a credit card.

Don't build it, import it

What's a module?

The investor's question has an answer: **you shouldn't build it from scratch.** Most of what you need is already written.

- A **module** is a toolbox of code someone already wrote and tested
- Python ships with a whole shelf of them: the **standard library**
- You `import` a module, then reach for the tools inside it with a dot: `math.ceil(...)`

Tobi has been hand-rolling arithmetic for months…

`import math`: stop rounding by hand

130 pastries need to ship. They go in crates of 48. How many crates? You need to round **up**: a half-full crate still ships as a whole one.

```
import math

pastries = 130
per_crate = 48
crates = math.ceil(pastries / per_crate)   # ceil = round UP
print(crates)
```

```
3
```

`130 / 48` is `2.7…`; `math.ceil` bumps it to **3**. No fiddling with "if there's a remainder, add one." The tool already knows.

`from statistics import ...`

Sometimes you only want a couple of tools, not the whole box. Import them **by name** and use them directly, no `statistics.` prefix:

```
from statistics import mean, median

ratings = [4.5, 4.8, 1.0, 5.0, 4.2]
print(mean(ratings))     # the average
print(median(ratings))   # the middle value
```

```
3.9
4.5
```

One furious review (a 1.0) drags the mean down to 3.9. The investor asked for the **typical** rating: `median` sorts the values and hands back the middle one (4.5), unmoved by one angry customer.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_06_a/



Any questions? Let me know!

Aliases: a shorter name

Some module names are long, or you'll type them fifty times. `import ... as` gives a module a **nickname** for the rest of the file:

```
import statistics as stats

print(stats.median([4.5, 4.8, 1.0, 5.0, 4.2]))
```

```
4.5
```

`stats.median` is the same tool as `statistics.median`, just less to type. You'll meet fixed conventions soon (`import pandas as pd`); using the community's nickname makes your code instantly readable to everyone else.

Predict: which name now exists?

Tobi imports **just** `pi` by name, then reaches for it the way he reached for `math.ceil`. The large pizza has a 15 cm radius. What happens?

```
from math import pi

radius = 15
print(math.pi * radius ** 2)
```

a) prints `706.858…`
b) a `TypeError`
c) a `NameError`

Answer: `pi` exists, `math` doesn't

```
from math import pi

radius = 15
print(math.pi * radius ** 2)
```

```
NameError: name 'math' is not defined
[31mNameError[39m[31m:[39m name 'math' is not defined
```

c) a `NameError`: `from math import pi` puts exactly one name on your desk, `pi`. Nobody ever created a name called `math`, so `math.pi` fails at the first dot.

Use the name you imported, and it works:

```
print(pi * radius ** 2)   # 706.858…: the imported name, no prefix
```

```
706.8583470577034
```

Looking inside a module

You don't have to memorize a module. Python will tell you what's in it and what each tool does:

```
import math

dir(math)          # lists every name in the module
help(math.ceil)    # prints what ceil does, and how to call it
```

> **Tip**
> `dir()` is the drawer of tools; `help()` is the little instruction card taped to each one. Between them you can explore any module without leaving your editor.

Predict: which way does floor go?

`math.ceil` rounds up. Its partner `math.floor` rounds **down**, but *down* from a negative number is the tricky part. What does the last line print?

```
import math
print(math.floor(-2.5))
```

a) `-3`
b) `-2`
c) an error

Answer: floor always goes down

```
import math
print(math.floor(-2.5))   # down the number line → -3
```

```
-3
```

a) `-3`: `floor` always heads **down** the number line, toward more negative. From `-2.5`, down is `-3`, not the `-2` you'd get by rounding toward zero.

"Down" means *smaller*, and `-3` is smaller than `-2`. Keep the number line in your head, not the distance to zero.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_06_b/

Any questions? Let me know!

Rehearsing luck

The `random` toolbox (1)

The investor wants to see how the shop copes with a **busy day**, but the busy day hasn't happened yet. So we *rehearse* it with made-up numbers. The `random` module deals them:

```
import random

print(random.random())                          # a float in [0.0, 1.0)
print(random.randint(1, 20))                    # an integer 1-20, ends included
print(random.choice(["latte", "mocha", "tea"]))  # one item, picked at random
```

```
0.9376883141892404
5
tea
```

The `random` toolbox (2)

```
import random

queue = [1, 2, 3, 4, 5]
random.shuffle(queue)                           # reorders the list in place
print(queue)
```

```
[5, 4, 1, 2, 3]
```

Four tools, four flavors of luck: a raw float, a bounded integer, a pick from a list, and a reshuffle.

"Run it again"

The investor leans over and says: **"Run it again."** Tobi does, and gets **completely different numbers**:

```
import random

print([random.randint(1, 20) for _ in range(5)])   # _ : a loop name we never
use
print([random.randint(1, 20) for _ in range(5)])   # ...and again, different!
```

```
[3, 13, 2, 15, 1]
[18, 4, 6, 5, 13]
```

Two runs, two answers. That's *exactly* what random is supposed to do, but it's useless for due diligence. A projection nobody can reproduce is a projection nobody can trust.

`random.seed` makes luck repeatable

`random.seed(n)` fixes the starting point of the number stream. Same seed → **same sequence, every time**:

```
import random

random.seed(7)
print([random.randint(1, 20) for _ in range(5)])   # → [11, 5, 13, 2, 3]

random.seed(7)
print([random.randint(1, 20) for _ in range(5)])   # same seed → same list


[11, 5, 13, 2, 3]
[11, 5, 13, 2, 3]
```

Both lines print `[11, 5, 13, 2, 3]`. The numbers still *look* random, but now the investor can run it herself and land on the identical result.

Seed, pick, seed again

The busiest day needs an extra rider. Tobi seeds, picks one, then seeds **again with the same number** and picks again. Is it the same rider?

Predict: same seed, same rider?

```
import random

riders = ["Ana", "Ben", "Cem", "Dana"]

random.seed(3)
first = random.choice(riders)
random.seed(3)
second = random.choice(riders)

print(first == second)
```

a) `False`: `choice` picks freely, the seed only steers `randint`
b) `True`: same seed, same stream, same pick
c) an error: a seed can only be set once per program

Answer: same rider

```
import random

riders = ["Ana", "Ben", "Cem", "Dana"]

random.seed(3)
first = random.choice(riders)
random.seed(3)
second = random.choice(riders)
```

```
print(first == second)
print(first, second)   # Ben Ben: the seed makes every tool in the box
repeatable


True
Ben Ben
```

b) `True`. Every `random` tool draws from the same stream, and re-seeding rewinds it to the start. `choice` is no exception: after `seed(3)` it lands on the same item every time.

Predict: seeded once, built twice

Tobi seeds **once**, then builds two lists the same way, without touching the seed in between. Are `first` and `second` equal?

```
import random

random.seed(42)
first  = [random.randint(1, 20) for _ in range(3)]
second = [random.randint(1, 20) for _ in range(3)]

print(first)
print(second)
```

a) different: the second list continues where the first stopped
b) equal: the seed is set, so both come out the same
c) an error: the stream is empty after three draws

Answer: different

```
import random

random.seed(42)
first  = [random.randint(1, 20) for _ in range(3)]
second = [random.randint(1, 20) for _ in range(3)]

print(first)    # [4, 1, 9]: the stream's first three numbers
print(second)   # [8, 8, 5]: the stream carries on


[4, 1, 9]
[8, 8, 5]
```

a) different. A seed doesn't freeze `random`, it fixes the whole sequence. The first list eats the first three numbers of the stream; the second list simply continues from number four.

To get the *same* list twice, re-seed before each run: it rewinds the stream. One seed, one fixed sequence: that's a seed's entire job.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_06_c/

Any questions? Let me know!

To the Lab

After the break: the lab

- Head to the lab notebook: Episode 6: Due Diligence Week
- You'll `import math` and `statistics` for investor-grade counts and averages, then use `random` with a `seed` to rehearse a busy day she can reproduce
- It's the **first lab where AI is allowed**, and the first without hint boxes: ask the AI to explain
- It runs entirely in your browser: no setup, just click and code

> ! **Important**
> **Download your `.py` before you leave.** Closing the tab without downloading loses your work, and downloading is exactly how you handed in the checkpoint at the start of the session.

Before the next session

- The **Mistral account** and the **Zed student plan** from the AI Tools page: together about ten minutes
- **We install Zed in class next session.** Bring a laptop you can install software on, and its charger

Wrap-up

Three things to remember

1. **Don't build it, import it.** A **module** is a toolbox someone already wrote: `import math`, `from statistics import median`, `import ... as` for a nickname. `dir()` and `help()` show you what's inside.
2. **`random` deals the luck** (`random()`, `randint`, `choice`, `shuffle`), perfect for rehearsing a day that hasn't happened yet.
3. **`random.seed(n)` makes luck repeatable.** Same seed → same sequence, every run. A projection you can reproduce is a projection an investor can trust.

> **Note**
> **Next episode: the numbers deck.** The shop's data has outgrown plain lists, and NumPy turns a thousand numbers into a single, fast object.

Literature

Books to start with

> **Note**
> New this session: the AI Tools page: how and when to use AI in Part II, and the disclosure section your project README needs in Part III.

For more, see the literature list of this course.

Bibliography